

8086指令系统 (2)





三、逻辑运算和移位类

逻辑运算 与，或，非，异或

移位操作

非循环移位

循环移位

除“非”，其余指令的执行
都会影响除AF的寄存器
无论执行结果如何，都会使 $OF = CF = 0$



3.1、逻辑运算

- 逻辑运算指令对操作数的要求大多与MOV指令相同。

1.“与”指令：

- 格式：

- **AND OPRD1, OPRD2**

- 操作：

- 两操作数相“与”，结果送目标地址。



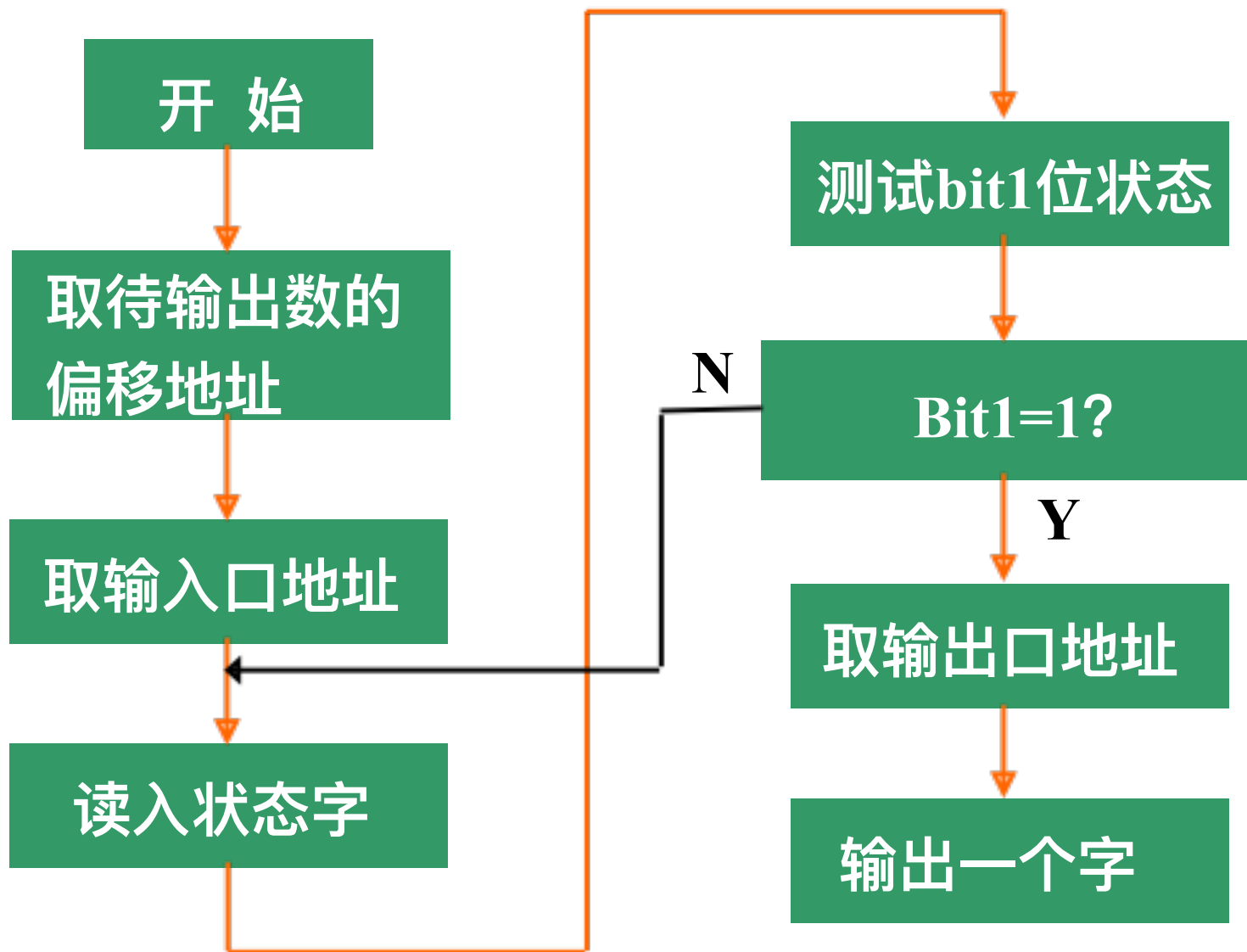
“与”指令的应用

- 实现两操作数按位相与的运算
 - `AND BL, [SI]`
- 使目标操作数的某些位不变，某些位清零
 - `AND AL, 0FH` 屏蔽
- 在操作数不变的情况下使CF和OF清零
 - `AND AX, AX`

852

例

从地址为3F8H的端口中读入一个字节数，如果该数的bit1位为1，则将DATA为首地址的一个字输出到38FH端口，否则就不能进行数据传送。



- 从地址为3F8H的端口中**读入**一个字节数，如果该数的bit1位为1，则将DATA为首地址的一个字**输出**到38FH端口，否则就不能进行数据传送。



LEA SI, DATA

MOV **DX**, 3F8H

(652由DX控制)

WATT: IN AL, DX

AND AL, 02H

JZ WATT

; ZF=1转移

MOV DX, 38FH

MOV AX, [SI]

OUT DX, AX

2.“或”运算指令

格式：

OR OPRD1, OPRD2



操作：

两操作数相“或”，结果送目标地址

实现两操作数相“或”的运算

OR AX, [DI]

使某些位不变，某些位置“1”

OR CL, 0FH

在不改变操作数的情况下使OF=CF=0

OR AX, AX



“或”指令的应用例

(1). **OR AL, AL**
 JPE GOON
 OR AL, 80H

GOON:

PF=1转移

将一个二进制数9变
为字符‘9’

如何实现？



3. “非”运算指令

- 格式：
 - NOT OPRD
- 操作：
 - 操作数按位取反再送回原地址
- 注：
 - 指令中的操作数不能是立即数
 - 指令的执行对标志位无影响
- 例：NOT BYTE PTR[BX]

1 字节长是



4. “异或”运算指令

- 格式：

- `XOR OPRD1, OPRD2`

- 操作：

- 两操作数相“异或”，结果送目标地址

- 例：

`XOR BL, 80H` 最高位取反

`XOR AX, AX` 清0

两操作数自身相异或，结果是0



5. “测试”指令

- 格式：

- **TEST OPRD1, OPRD2**

- 操作：

- 执行“与”运算，但运算的结果不送回目标地址。
区别于 AND 指令

- 应用：

- 常用于测试某些位的状态

从地址为3F8H的端口中读入一个字节数，当该数的bit1，bit3， bit5位同时为1时，可从38FH端口将DATA为首地址的一个字输出，否则就不能进行数据传送。

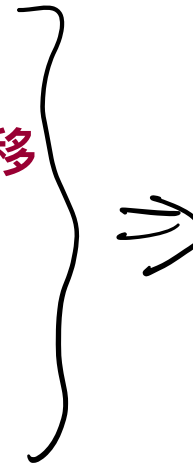


```
LEA SI, DATA
MOV DX, 3F8H
```

```
WATT: IN AL, DX
      TEST AL, 02H
      JZ WATT
      TEST AL, 08H
      JZ WATT
      TEST AL, 20H
      JZ WATT
      MOV DX, 38FH
      MOV AX, [SI]
      OUT DX, AX
```

× × (×) × (×) × (×) ×
0 0 1 0 1 0 1 0

; ZF=1转移



```
AND AL, 2AH
CMP AL, 2AH
JNZ WATT
```



3.1、逻辑运算

- 逻辑运算指令对操作数的要求大多与MOV指令相同。
- “非”运算指令要求操作数不能是立即数；
- 除“非”运算指令外，其余指令的执行都会使标志位 $OF=CF=0$



3.2、移位指令

非循环移位指令

循环移位指令



- 逻辑左移
- 算术左移
- 逻辑右移
- 算术右移

算术左移和逻辑左移



CF

0

- 算术左移指令:

SAL OPRD, 1

SAL OPRD, CL

有符号数

- 逻辑左移指令:

SHL OPRD, 1

SHL OPRD, CL

无符号数

注:

- 移动一位时由指令直接给出;
- 移动两位及以上, 则移位次数由CL指定;
- 移动一位后, 若CF与最高位不相等, 则OF=1; 否则OF=0 SAL时

移位次数是1或CL



逻辑右移

- 格式:

SHR OPRD, 1

SHR OPRD, CL

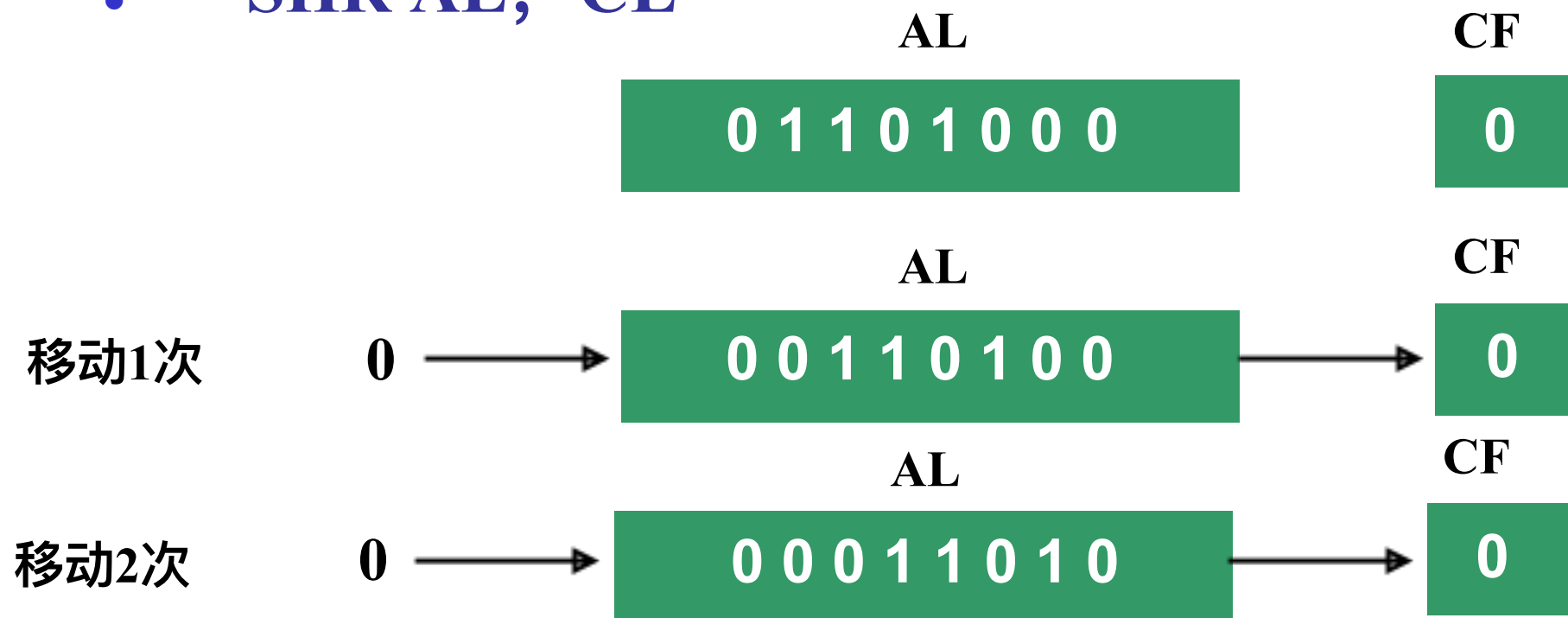
无符号数
的右移





逻辑右移例：

- `MOV AL, 68H`
- `MOV CL, 2`
- `SHR AL, CL`



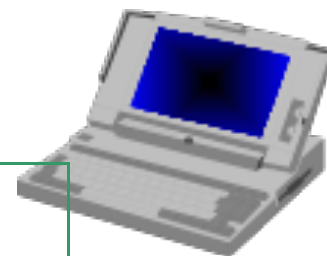
算术右移

格式:

SAR OPRD, 1

SAR OPRD, CL

有符号数
的右移



右移完后全部数据均变为与符号
位相同的数据。

非循环移位指令的应用

- 左移可实现乘法运算
- 右移可实现除法运算



程序例

```
MOV SI,1000H
MOV DI,3000H
MOV CX,4
BBB: MOV AL,[SI]
      MOV BL,AL
      AND AL,0FH
      OR AL,30H
      MOV [DI],AL
      INC DI
      MOV AL,BL
```

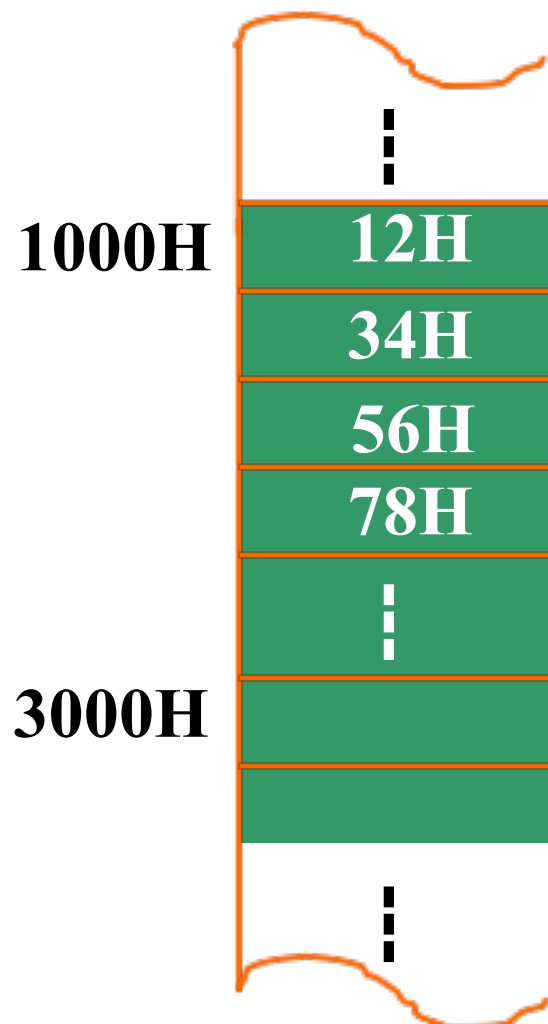
```
PUSH CX
MOV CL,4
SHR AL,CL
OR AL,30H
MOV [DI],AL
INC DI
INC SI
POP CX
DEC CX
JNZ BBB
HLT
```

程序
功能?



程序功能

- 将1000H开始存放的四个压缩BCD码转换为ASCII码存放在3000H开始的单元中去。





循环移位指令

- 不带进位的循环移位

左移 ROL

右移 ROR

- 带进位的循环移位

左移 RCL

右移 RCR

指令格式、对操作数的要求与非循环移位指令相同



不带进位的循环移位

ROL OPRD, 1



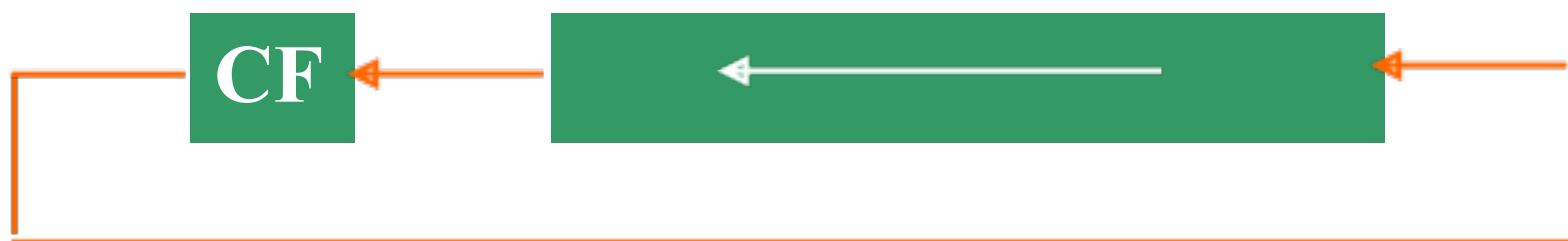
ROR OPRD, 1



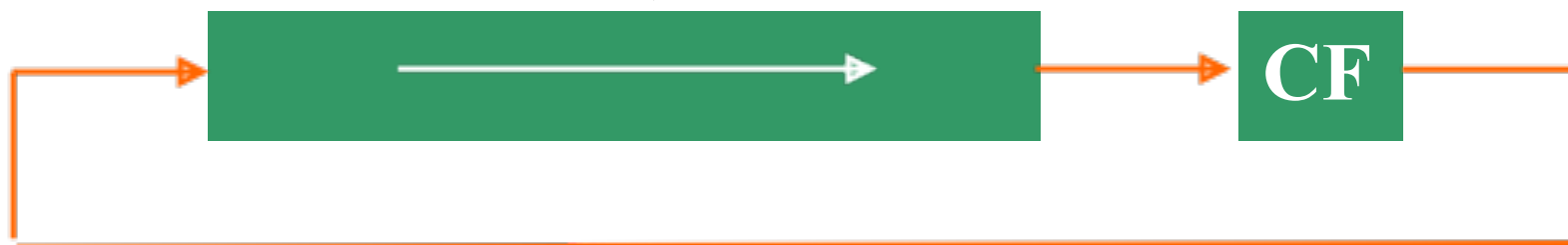


带进位位的循环移位

RCL OPRD, 1



RCR OPRD, 1





循环移位指令的应用

- 用于对某些位状态的测试；
- 高位部分和低位部分的交换；
- 与非循环移位指令一起组成32位或更长字长数的移位。



例：

- 计算 $2658+3619=?$
结果存放在SUM单元
- 说明：
 - 被加数和加数的每一位都以ASCII码形式存放在内存中，低位在前，高位在后。

STRING1

38H

35H

36H

32H

STRING2

...

39H

31H

36H

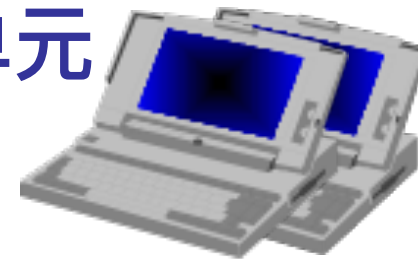
33H

SUM

...

数据段

- 计算 $2658+3619=?$ 结果存放在SUM单元



代码1:

LEA SI, STRING1 ; 取偏移地
址

LEA DI, STRING2

LEA BX, SUM

MOV CH, 4 ; 设循环次
数

CLC ; 使CF清零

A: AND BYTE PTR[SI], 0FH ;
将ASCII转换为二进制数

AND BYTE PTR[DI], 0FH

MOV AL, [SI] ; 两字节数
求和

ADC AL, [DI]

MOV DL,AL

AND AL,0FH

OR AL,30H

MOV [BX],AL

INC BX

MOV AL,DL

MOV CL,4

SHR AL,CL

OR AL,30H

MOV [BX],AL

INC BX

INC SI

INC DI

DEC CH

JNZ A

HLT

(1) 非压缩型BCD码加法调整指令AAA (ASCII Adjust for Addition)

指令格式:

AAA

AAA也称为加法的ASCII调整指令。指令后面不写操作数，但实际上隐含累加器操作数AL和AH。指令的操作为:

如果 $(AL) \wedge 0FH > 9$, 或 $(AF) = 1$

则 $(AL) \leftarrow (AL) + 06H$

$(AH) \leftarrow (AH) + 1$

$(AF) \leftarrow 1$

$(CF) \leftarrow (AF)$

$(AL) \leftarrow ((AL) \wedge 0FH)$

否则 $(AL) \leftarrow ((AL) \wedge 0FH)$

由上可见，指令将影响AF和CF标志，但状态标志位SF、ZF、PF和OF的状态不确定。

LEA BX, SUM

MOV CX, 4

CLC

NEXT: MOV AL, [SI]

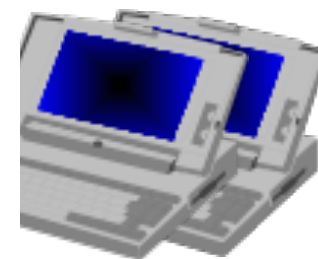
ADC AL, [DI]

AAA

OR AL, 30H

MOV [BX], AL

DEC CX
JNZ NEXT
HLT





串操作指令



串操作指令说明

- 针对数据块或字符串的操作；
- 可实现存储器到存储器的数据传送；
- 待操作的数据串称为源串，目标地址称为目标串。

- 串传送 MOVS
- 串比较 CMPS
- 串扫描 SCAS : 在一串数据中查找关键字
- 串装入 LODS
- 串送存 STOS

1. 串传送指令



格式:

MOVS OPRD1, OPRD2

MOVSB 按字节传送

MOVSW 按字传送

ES: DI

DS: SI

串传送指令常与无条件重复前缀REP连用

当CX ≠ 0时, REP指令将

继续重复执行

串操作指令的特点



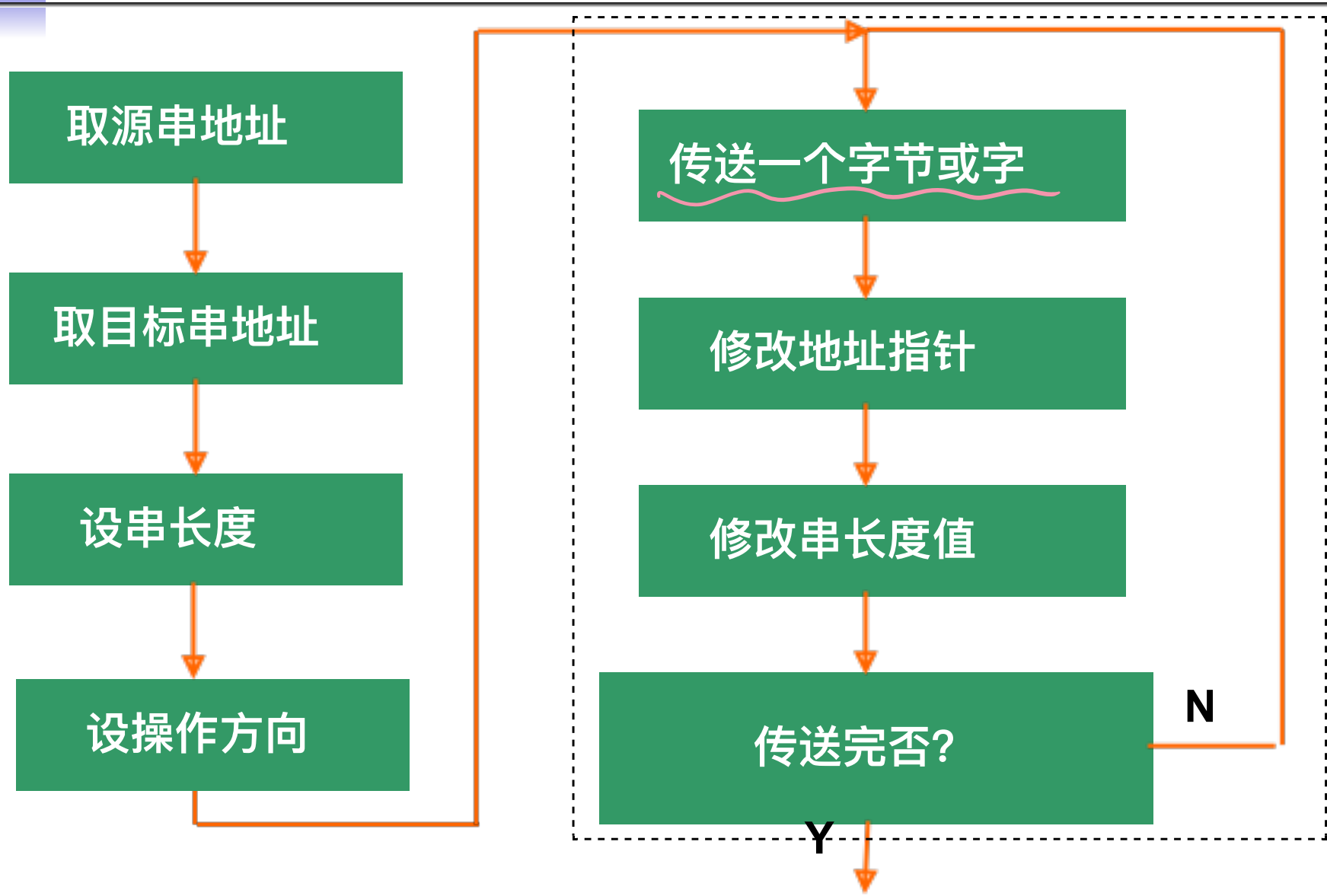
MOVS OPRD1, OPRD2
MOVSB
MOVSW

- 源串一般存放在**数据段**，**偏移地址由SI指定**。允许段重设；
- 目标串必须在**附加段**，**偏移地址由DI指定**；
- 指令**自动修改**地址指针，**修改方向由DF决定**。
 - DF=0 → 增地址方向；
 - DF=1 → 减地址方向；
-  数据块**长度值**由**CX指定**
- 可增加**自动重复前缀**以实现自动修改CX内容。

REP MOVSB



串操作指令流程



串传送指令



• 对比用MOV指令和MOVS指令实现
将200个字节数据从内存的一个区域送到
另一个区域的程序段。

• LEA SI, MEM1
 LEA DI, MEM2
 MOV CX, 200
 CLD 将DF清零
 REP MOVSB
 HLT



2. 串比较指令

- 格式：

CMPS OPRD1, OPRD2

CMPSB

CMPSW

按字节比较
按字比较
(运算类指令)

- 串比较指令常与条件重复前缀连用，指令的执行不改变操作数，仅影响标志位



重复前缀

- 无条件重复

- **REP** → **CX≠0 重复**

- 条件重复

- **REPE** 相等重复

- **REPZ** 为零重复

- **REPNE** 不相等重复

- **REPNZ** 不为零重复

CX≠0 ZF=1

CX≠0 ZF=0



串比较指令例

测试200个字节数据是否传送正确:

LEA SI, MEM1

LEA DI, MEM2

MOV CX, 200

CLD

REPE CMPSB

TEST CX, 00FFH

JZ STOP

DEC SI

MOV AL, [SI]

MOV BX, SI

STOP: HLT

ZF=1 对

指向存放数据的地址

*结束比较指令的条件
① CX=0 ② CX≠0, ZF=0*



3. 串扫描指令

目 标
操作数

格式：

SCAS OPRD

SCASB *字节* → AL

SCASW *字* → AX

- 执行与CMPS指令相似的操作，只是这里的源操作数是AX或AL
- 常用于在指定存储区域中寻找某个关键字。

串扫描指令应用例：

- 在ES段中从2000H单元开始存放了10个字符，寻找其中有无字符“A”。若有则记下搜索次数，将搜索次数写入到DATA1单元，并将存放“A”的地址写入DATA2单元。

```
MOV DI, 2000H
MOV BX, DI    将偏移地址备份
MOV CX, 0AH   10个字符
MOV AL, 'A'
```

```
CID
```

```
REPNE SCASB
```

不相等则重复前指令

```
JZ FOUND
```

```
MOV DI, 0
```

```
JMP DONE
```

CF=0 或 ZF=1
(两种情况)

```
FOUND: DEC DI
```

```
MOV DATA2, DI
```

```
INC DI
```

```
SUB DI, BX    得到搜索次数
```

```
DONE: MOV DATA1, DI
```

```
HIT
```



4. 串装入指令

格式：

源操作数

LODS OPRD

LODSB

LODSW

操作：

对字节：**AL**

[DS:SI]

对字：**AX**

[DS:SI]

• 用于将内存某个区域的数据串依次装入累加器，以便显示或输出到接口。

• **LODS指令一般不加重复前缀。**



5. 串存储指令

目 标
操作数

- 格式：

STOS OPRD

STOSB

STOSW

- 操作：

- 对字节： **AL**

- 对 字： **AX**

→ **[ES:DI]**

→ **[ES:DI]**



串存储指令例——内存某个区域清零

- 设计思想：





程序控制指令

- 转移指令
- 循环控制
- 过程调用
- 中断控制



一、转移指令

通过修改指令的偏移地址或段地址及偏移地址
实现程序的转移

无条件转移指令

无条件转移到目标地址，执行新的指令

有条件转移指令

在具备一定条件的情况下转移到目标地址



1. 无条件转移指令

- 格式：

JMP OPRD

目标地址

与JMP在同
一段代码段

与JMP不在同
一段代码段

原则上可实现在整个内存空间的转移



段内直接转移

段内直接转移

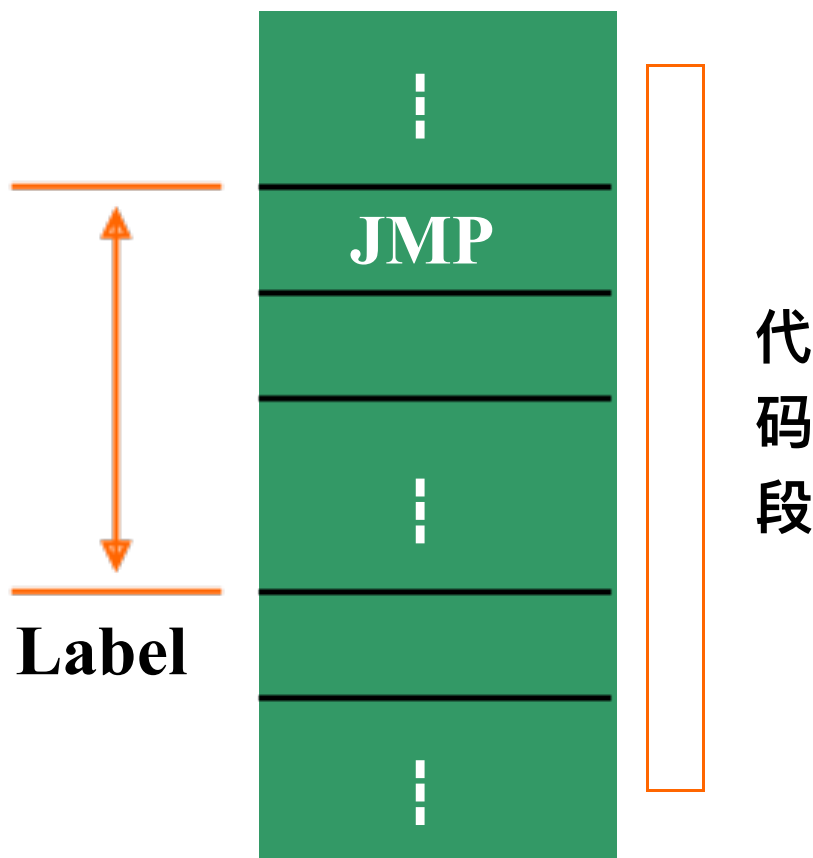
转移的目标地址由指令直接给出

格式:

JMP Label

位移量

近地址标号



下一条要执行指令的偏移地址=当前IP+位移量



段内间接转移

- 段内间接转移
 - 转移的目标地址存放在某个16位寄存器或存储器的某两个单元中

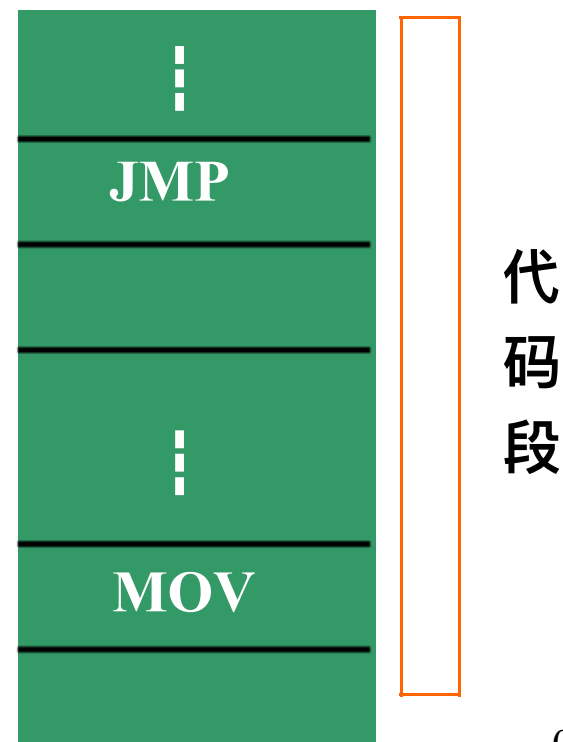
例：

JMP BX

若： **BX=1200H**

则： 转移的目标地址=1200H

1200H

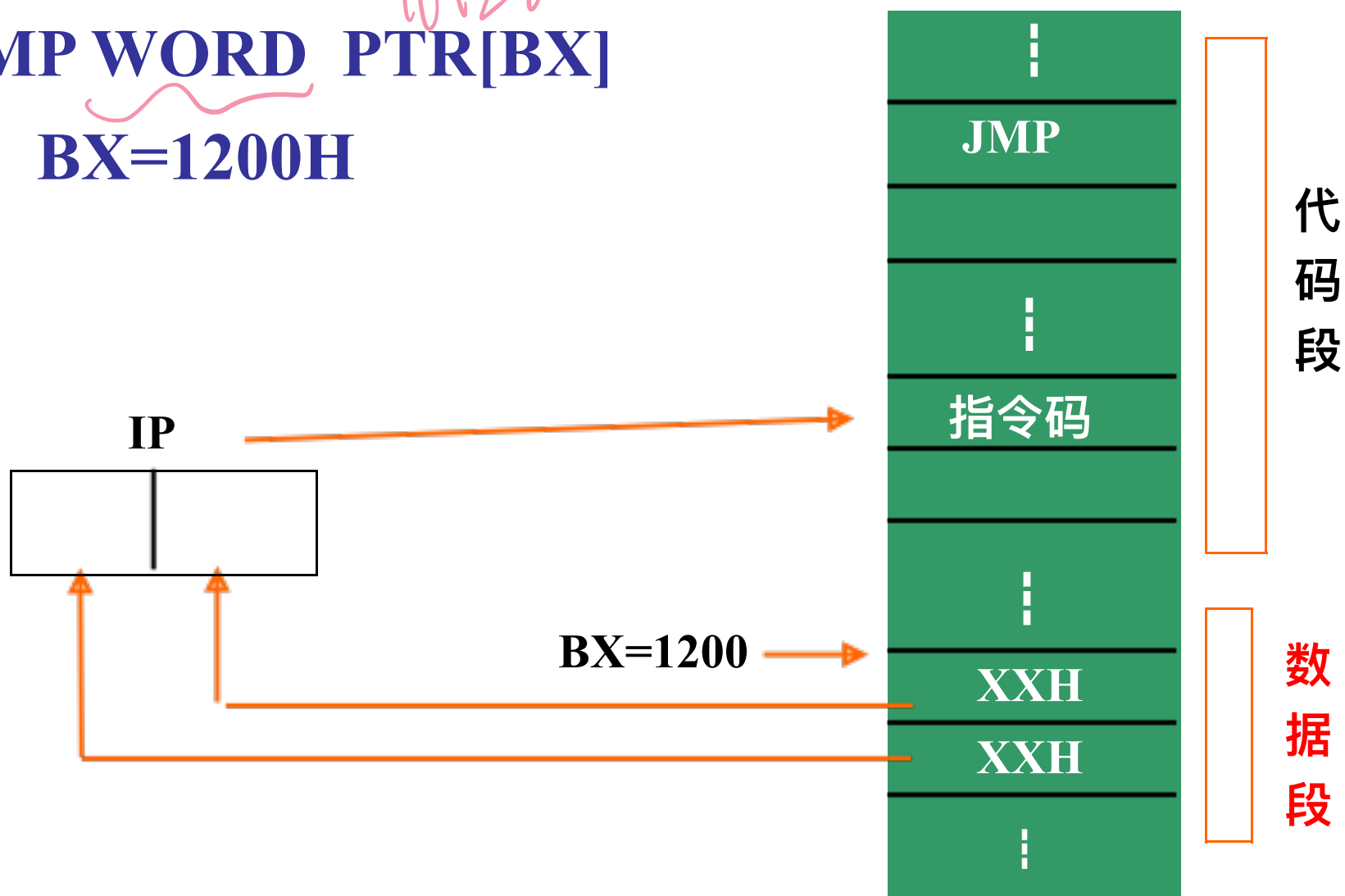




段内间接转移例

• **JMP WORD PTR[BX]**

设: **BX=1200H**





2. 条件转移指令

- 在满足一定条件下，程序转移到目标地址继续执行
- 条件转移指令均为段内短转移，即转移范围为：

-128-----+127

很小

■ 基于1个标志位状态实现转移的指令

■ JC/JNC

- 判断CF的状态。常用于两个无符号数大小比较

■ JZ/JNZ

- 判断ZF的状态。常用于循环体的结束判断

■ JO/JNO

- 判断OF的状态。常用于有符号数溢出的判断

■ JP/JPE、JNP/JPO

- 判断PF的状态。用于判断运算结果低8位中1的个数是否为偶数

■ JS/JNS

- 判断SF的状态。常用于判断数的性质

条件转移指令

■ 基于2个或3个标志位状态实现转移的指令：

■ JA/JAE/JB/JBE

- 判断CF或CF+ZF的状态。常用于无符号数大小的比较

■ JG/JGE/JL/JLE

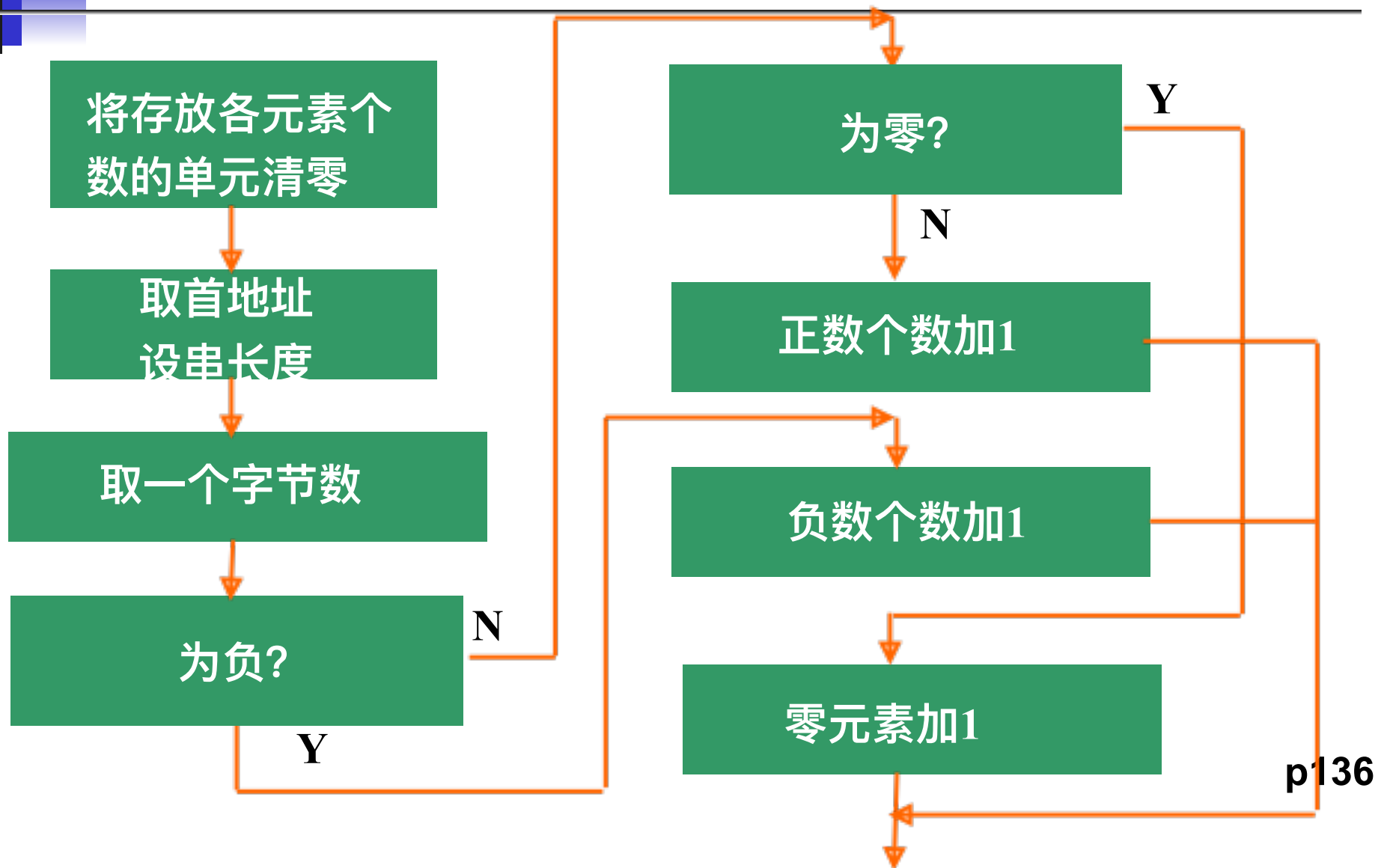
- 判断SF+OF或SF+OF+ZF的状态。常用于有符号数大小的比较

■ 基于CX内容转移的指令

■ JCXZ

- 可根据指令执行后CX的结果实现转移

- 统计内存数据段中以TABLE为首地址的100个8位符号数中正数、负数和零元素的个数。



START: XOR AL, AL

MOV PLUS, AL

MOV MINUS, AL

MOV ZERO, AL

LEA SI, TABLE

MOV CX, 100

CWD

CHECK: LODSB ; 串装入

OR AL, AL ; 除AF外, 其余5个状态标志位均改变

JS X1

JZ X2

INC PLUS

JMP NEXT

X1: INC MINUS

JMP NEXT

X2: INC ZERO

NEXT: DEC CX

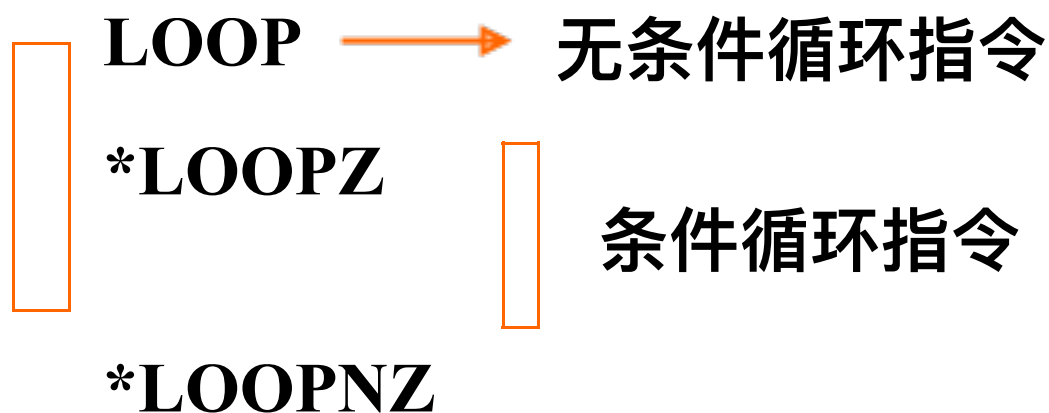
JNZ CHECK

HLT



二、循环控制指令

- 循环范围：
 - 以当前IP为中心的-128~+127范围内循环。
- 循环次数由CX寄存器指定。
- 循环指令：





无条件循环指令

- 格式:

LOOP LABEL

- 循环条件:

$CX \neq 0$

- 操作:

相当于 { **DEC CX**
JNZ 符号地址

条件循环指令

先 CX-1, 再根据 CX 中的值
及 ZF 值判断



- 格式:

LOOPZ LABEL

LOOPE LABEL

- 循环条件:

CX $\neq$ 0 ZF=1

- 格式:

LOOPNZ LABEL

LOOPNE LABEL

- 循环条件:

CX $\neq$ 0 ZF=0

例：

- 在以DATA为首地址的内存数据段中，存放有200个16位有符号数，试找出其中最大和最小的符号数，并分别放在MAX和MIN为首的内存单元中。

■ 题目分析：

- 先取数据块中第1个数，将其同时暂存于MAX和MIN中；
- 循环读取其它数并分别与MAX和MIN中的数进行比较
 - 若大于则取代原MAX中的数；若小于MIN内容，则将新数放于MIN中。直到结束。

注：有符号数比较应采用JG和JL等用于符号数的条件转移指令

bilibili

5

START: LEA SI, DATA

MOV CX, 200

CID

LODSW

MOV MAX, AX

MOV MIN, AX

DEC CX

NEXT:

LODSW

会自动读取4字节

CMP AX, MAX

修改指针

JG LARGE

CMP AX, MIN

JL SMALL

JMP GOOD

LARGE: MOV MAX, AX

JMP GOOD

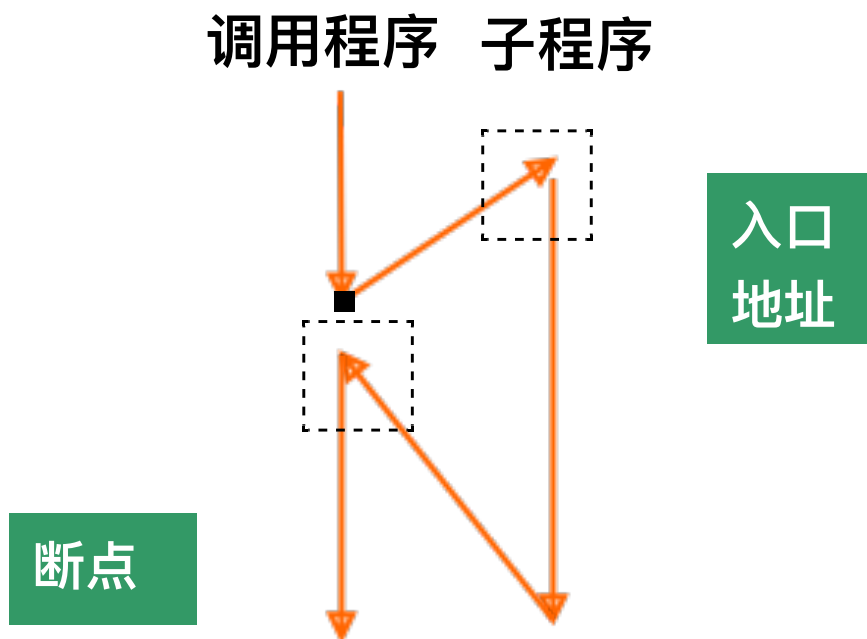
SMALL MOV MIN, AX

GOOD: LOOP NEXT

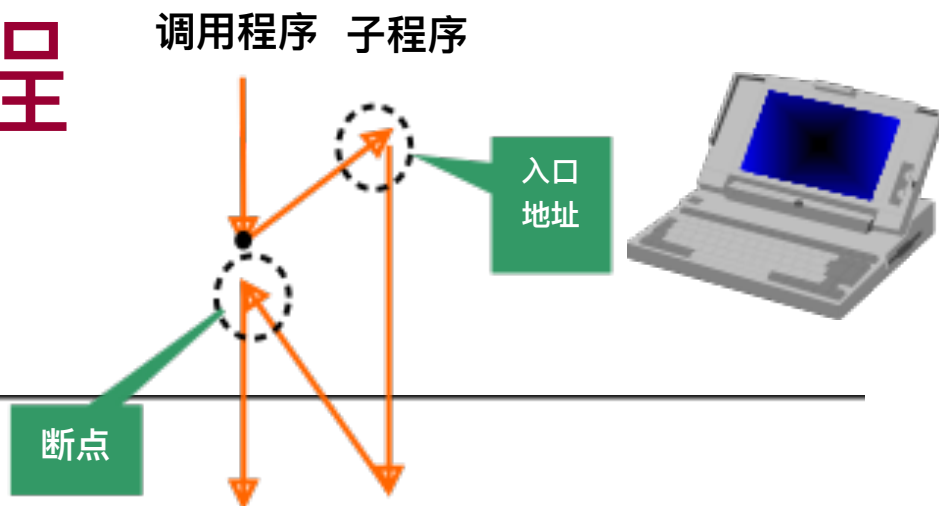
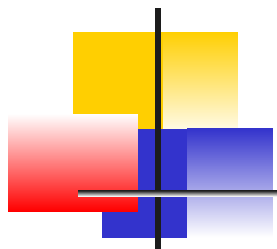


三、过程调用和返回

- 用于调用一个子过程；
- 子过程由程序员预先设计并装入内存 → 入口地址
- 子过程执行结束后要返回原调用处 → 断点地址



调用指令的执行过程



- 保护断点；
 - 将调用指令的下一条指令的地址（断点）压入堆栈
- 获取子过程的入口地址；
 - 子过程第1条指令的偏移地址
- 执行子过程，含相应参数的保存及恢复；
- 恢复断点，返回原程序。
 - 将断点偏移地址由堆栈弹出

过程调用



段内调用

段内直接调用

段内间接调用

段间调用

段间直接调用

段间间接调用

。直接：指令中直接给出子程序入口地址

。间接：由内存获得子过程的入口地址



1. 段内调用

- 被调用程序与调用程序在同一代码段
- 调用前只需保护断点的偏移地址

格式：

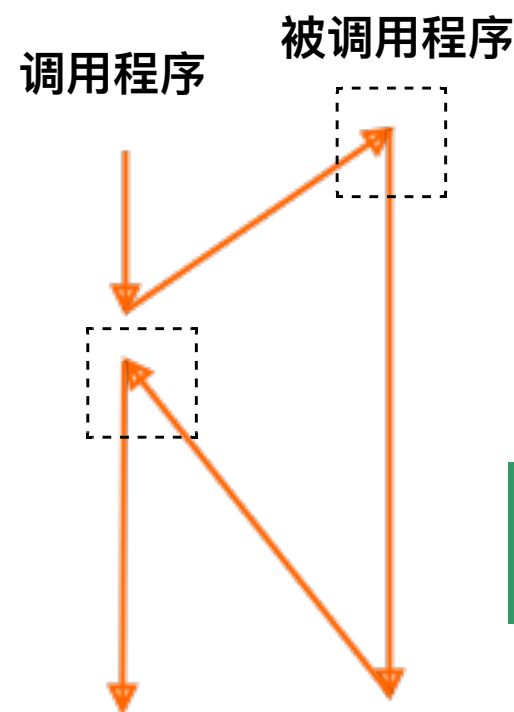
CALL NEAR PROC

近过程名

代码
段1

执行过程：

- 将断点的偏移地址压入堆栈
- 根据过程名找子程序入口





段内调用例

(1) CALL TIMRE $\longrightarrow$ 直接调用

(2) CALL WORD PTR[SI] $\longrightarrow$ 间接调用

设: SI=1200H
CS=6000H

执行第 (2) 条指令后:

CS = 6000H

IP = 3344H

与数据段相关

1200H

CALL

⋮

44H

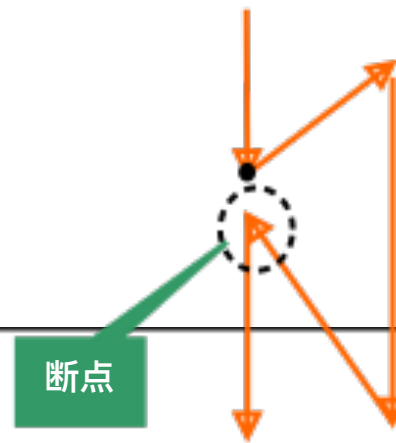
33H

代码段

数据段



3. 返回指令



- 功能：

- 从堆栈中弹出断点地址，返回原程序

- 格式：

RET

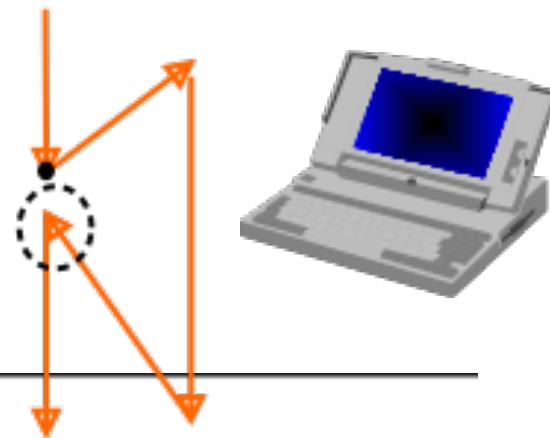
-  **RET指令一般位于子程序的最后。**



四、中断指令

- 中断
- 中断源
- 中断的类型
- 中断指令
- 引起CPU产生一次中断的指令

中断与过程调用：



- 中断是随机事件或异常事件引起，调用则是事先已在程序中安排好；
- 响应中断请求不仅要保护断点地址，还要保护FLAGS内容；
- 调用指令在指令中直接给出子程序入口地址，中断指令只给出中断向量码，入口地址则在向量码指向的内存单元中。

运行结果
指向的
地址



1. 中断指令

格式:

INT n

中断类型码

n=0 ~ 255

说明:

n x 4

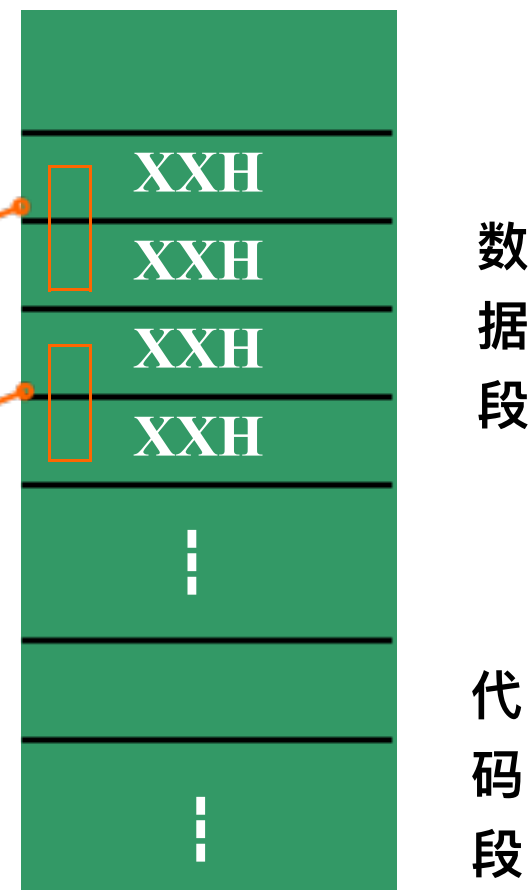
n x 4

入口的偏移地址

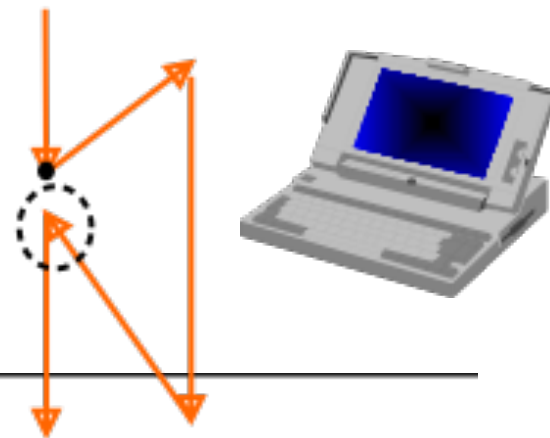
入口的段地址

存放中断服务子程序入口
地址单元的偏移地址

该单元在数据段, 段地址=DS



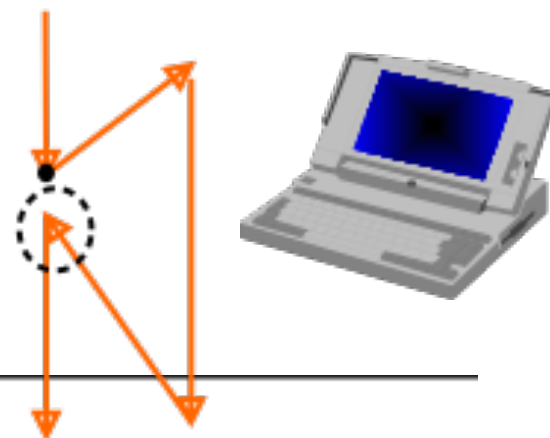
中断指令的执行过程



- 将FLAGS压入堆栈；
- 将INT指令的下一条指令的CS、IP压栈；
- 由 $n \times 4$ 得到存放中断向量的地址；
- 将中断向量（中断服务程序入口地址）送CS和IP寄存器；
- 转入中断服务程序。

段基址 转移地址

中断指令的执行过程



堆
栈
段



$n \times 4$

IP



CS



68122H

22H

11H

00H

67H

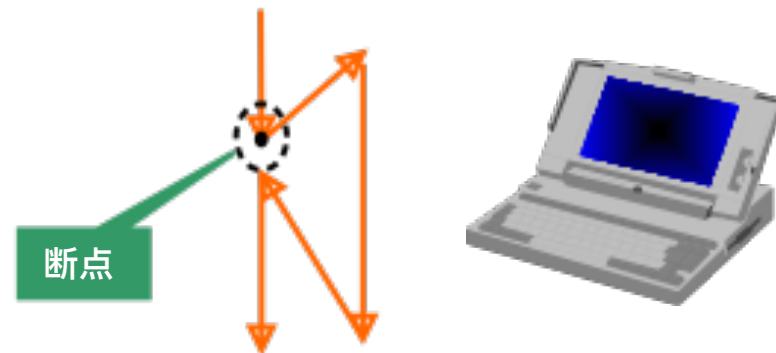
⋮

MOV

数
据
段

代
码
段

中断指令例



执行INT指令后

执行程序段:

CS IP

⋮

6200H:0110H INT 21H

6200H:0112H MOV AX, BX

⋮

SP=11FA



12H

01H

00H

62H

FLAGSL

FLAGSH

SP=1200



堆
栈
段

中断指令例

中断子程序

入口地址



- 执行INT 21H指令后

$IP = [21H \times 4]$

$CS = [(21H \times 4) + 2]$

0084H

IP

CS

23H

11H

00H

20H

数据段

21123H

XX

代码段

中断服务子程序



2. 溢出中断指令

- 格式:

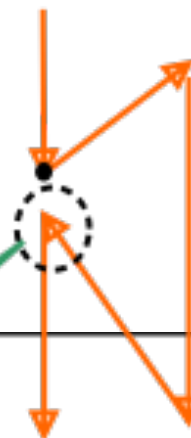
INTO $\xrightarrow{\text{相当于}}$ INT 4

- 若OF=1,则启动一个类型为4的中断过程,给出一个出错标志,如果OF=0,不做任何操作。
- INTO指令通常安排在有符号数加减运算指令之后。

3. 中断返回指令



断点



- 格式:

IRET

- 中断服务程序的最后一条指令，负责

恢复断点

恢复标志寄存器内容



六、处理器控制指令



控制对象是CPU
均为0 操作数格式

对标志位的操作

与外部设备的同步

说明见
p143表

CLC $CF \leftarrow 0$

STC $CF \leftarrow 1$

CMC $CF \leftarrow \overline{CF}$

CLD $DF \leftarrow 0$

STD $DF \leftarrow 1$

CLI $IF \leftarrow 0$

STI $IF \leftarrow 1$

串操作从低地址到高地址

串操作从高地址到低地址



本章课外自学内容:

- BCD码调整指令
- 远地址指针指令LDS, LES
- 条件循环指令LOOPZ, LOOPNZ



结束语：

掌握：

- 指令的格式及意义；
- 指令对操作数的要求及对标志位的影响；
- 指令的应用。